

15,000

workout sessions

0.99995

R^2 on the test set

0.96

kcal mean absolute error

Calories burnt prediction

- 01** Problem statement
- 02** Data understanding and quality
- 03** Feature engineering
- 04** Handling outliers
- 05** Leakage-safe pipeline
- 06** Feature selection
- 07** Modeling, tuning and robustness
- 08** Interpretation: importance and SHAP
- 09** Results, diagnostics and conclusion

Table of Content

Problem statement

- Fitness trackers often estimate calories with one-size-fits-all formulas based on exercise type and duration.
- A model trained on real session data can use heart rate, body temperature, age and body measurements to give a **more personal estimate**.
- The task is formulated as a **regression problem**: predict the calories burned in one workout session.

Task

Predict the calories burned in one session from seven inputs.

Inputs

Gender, age, height, weight, duration, heart rate, body temperature.

Success

Clearly beat the mean baseline ($R^2 = 0$) and keep MAE small versus the 1–314 kcal range.

Data understanding

15,000 workout sessions from two files, exercise.csv and calories.csv, joined on User_ID.

No data quality issues were found.

- No missing values, duplicates or impossible readings.
- No zeros hiding missing values.
- Gender is balanced: 50.4% female, 49.6% male.

Target distribution

- Calories is moderately right-skewed (0.51), so no log transform was needed for tree models.

Quality checks

Dimension	Check	Result
Completeness	Missing values	0
Validity	Out-of-range readings	0
Uniqueness	Duplicate rows / IDs	0 / 0
Hidden missing	Zero values	0
Balance	Female / male	50.4 / 49.6%

Feature engineering

Workload

Duration × Heart_Rate: total effort, not just length or intensity.

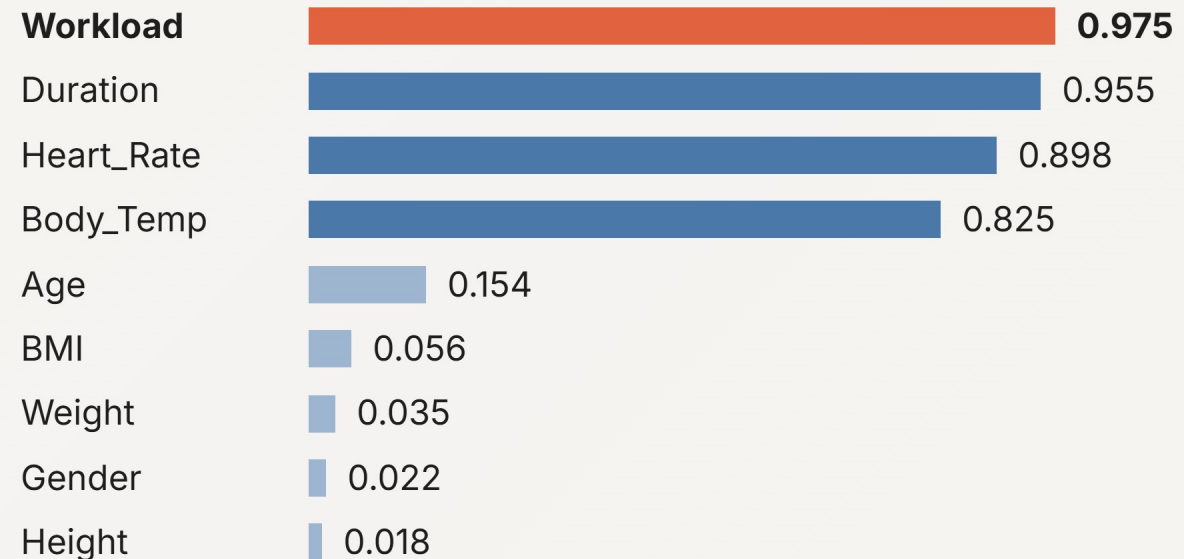
BMI

Weight / (Height / 100)².

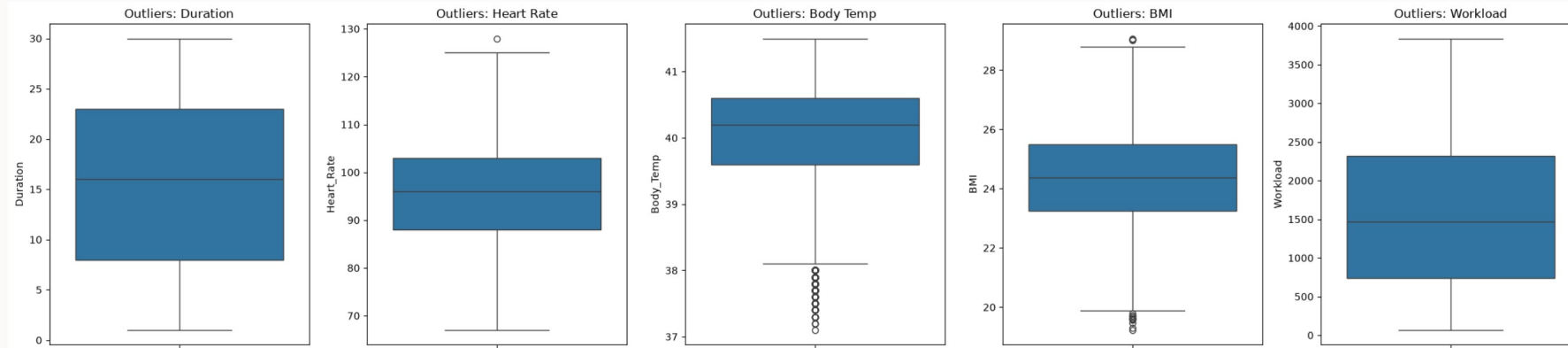
Gender → 0/1

Two categories, so one column equals one-hot. User_ID dropped.

Pearson correlation with Calories



Handling outliers



What the boxplots show

Body_Temp: low tail. Heart_Rate: one outlier point (128 bpm). BMI: both tails.

Why IQR, not 3σ

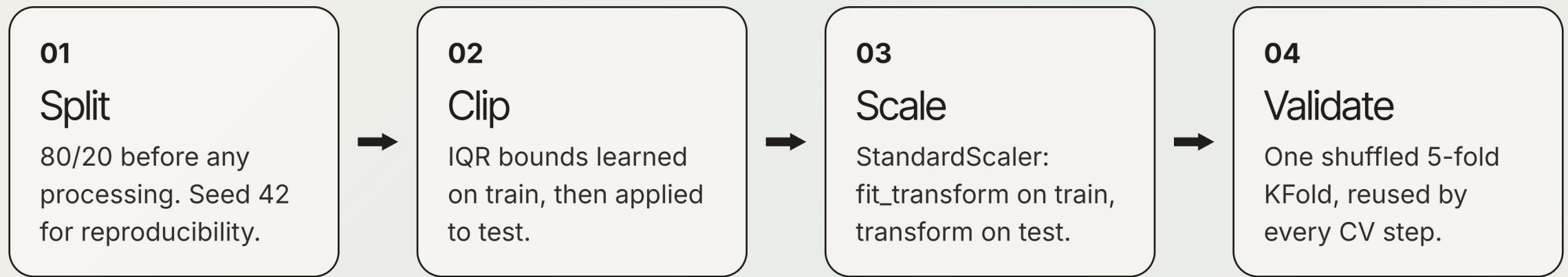
Tukey's $k = 1.5$ needs no normality, and quartiles are robust to the extremes.

Why clip, not delete

Values passed plausibility checks: rare but real. Clipping keeps every row.

Leakage-safe pipeline

Every preprocessing step is **fit on the training set only** and then applied to the test set.



Why: if test rows shaped the bounds or the scaling, information would leak into training and the test score would be too optimistic.

Pipeline in code

1 · Split before any processing

```
X = df.drop(columns=["Calories", "User_ID", "Gender"])
y = df["Calories"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=RANDOM_STATE)
X_train, X_test = X_train.copy(), X_test.copy()
```

2 · Clip outliers with IQR bounds learned on train only

```
columns_with_outliers = ["Heart_Rate", "Body_Temp", "BMI"]
for col in columns_with_outliers:
    lower, upper = compute_iqr_bounds(X_train[col])
    X_train = clip_with_bounds(X_train, col, lower, upper)
    X_test = clip_with_bounds(X_test, col, lower, upper)
```

3 · Scale: fit on train, only transform on test

```
preprocessor = ColumnTransformer(transformers=[("num", StandardScaler(), numeric_features),])
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

Feature selection

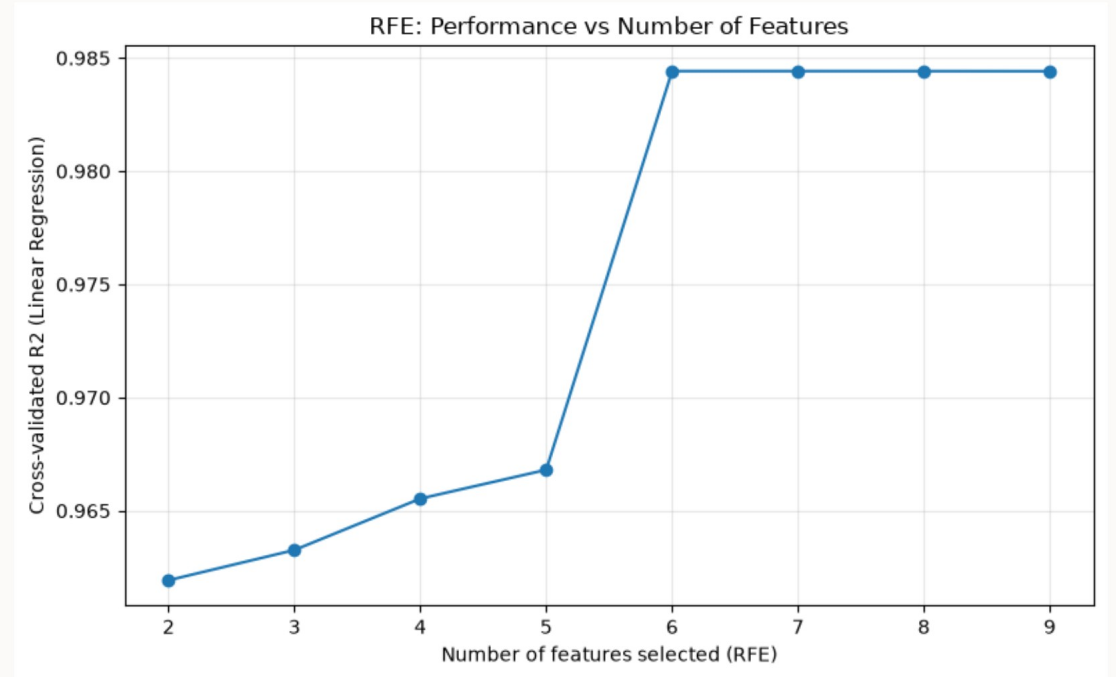
Two methods were combined.

- **Filter:** correlation with the target, computed on the training set only.
- **Wrapper:** RFE with linear regression drops the weakest feature step by step, scored by 5-fold CV.

Result: all nine features were kept.

- CV R^2 jumps from 0.967 at 5 features to 0.984 at 6, then stays flat to 9.
- With only 9 features, RFE is a diagnostic, not a filter. No PCA needed.

RFE: CV R^2 vs number of features



Modeling approach

- Six model families were compared, from **simple linear baselines** to **tree ensembles**, all on the same 5 folds.
- R^2 and RMSE are the main metrics; MAE is reported because it reads directly in kcal.

Polynomial, degree 2

0.9845 $\rightarrow$ 0.9987

Linear Regression R^2 with squares and products added.
MAE 5.83 $\rightarrow$ 1.72 kcal: the relation has curvature.

Non-linear models clearly beat linear ones; Gradient Boosting is best from the start.

Metrics + comparison

Model	Test R^2	RMSE	MAE	5-fold CV R^2
Gradient Boosting	0.9979	2.91	2.13	0.9974 $\pm$ 0.0001
Random Forest	0.9953	4.36	3.04	0.9948 $\pm$ 0.0001
k-NN (k = 15)	0.9946	4.65	3.33	0.9936 $\pm$ 0.0003
Linear Regression	0.9845	7.90	5.83	0.9844 $\pm$ 0.0003
Ridge	0.9845	7.91	5.81	0.9844 $\pm$ 0.0003
Decision Tree (d = 6)	0.9818	8.58	6.06	0.9807 $\pm$ 0.0004

Hyperparameter tuning

Tuning lifted every model; **Gradient Boosting reached R^2 0.9995** on the test set.

Model	Search	Best parameters	Baseline	Tuned
Gradient Boosting	Random, 5 of 8	400 trees, depth 5, learning rate 0.1	0.9979	0.9995
Random Forest	Random, 5 of 6	250 trees, depth 10	0.9953	0.9974
k-NN	Grid, all 10	k = 15, distance weights	0.9946	0.9950

How each candidate was scored

5-fold cross-validation on the training set only, R^2 as the score; the best model is refit on all training data.

Best values sit at the grid edge

GB uses the largest values in its grid (400 trees, depth 5), RF the largest depth (10): better values may exist beyond the search space.

Tuning in code

1 • Define the search space: $2 \times 2 \times 2 = 8$ combinations

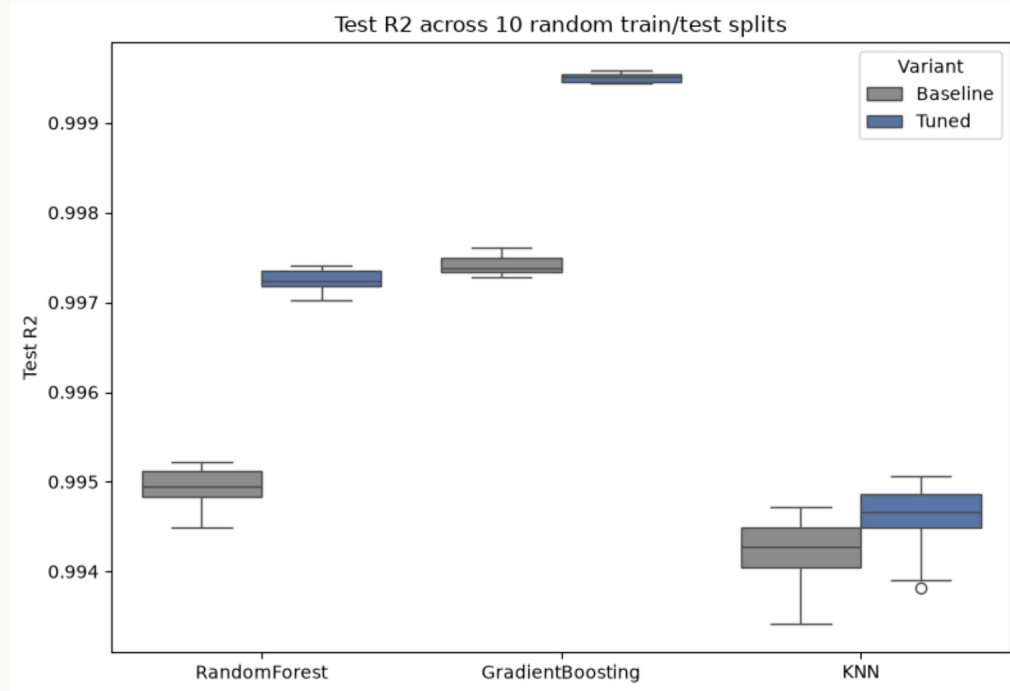
```
gb_param_grid = {  
    "n_estimators": [200, 400],  
    "learning_rate": [0.05, 0.1],  
    "max_depth": [3, 5],  
}
```

2 • Try 5 of them, score each by 5-fold CV on train; the best is refit on all training data

```
gb_search = RandomizedSearchCV(  
    estimator=GradientBoostingRegressor(random_state=RANDOM_STATE),  
    param_distributions=gb_param_grid,  
    n_iter=5,  
    scoring="r2",  
    cv=cv,  
    random_state=RANDOM_STATE,  
    n_jobs=-1,  
)  
gb_search.fit(X_train_processed, y_train)
```

Robustness check

Test R^2 over 10 random splits



Tuning used one 80/20 split, so the data was **re-split 10 times** and both configurations retrained.

Mean R^2 gain, tuned – baseline (95% CI)

+0.0021

Gradient Boosting · [0.0020, 0.0022]

+0.0023

Random Forest · [0.0022, 0.0024]

+0.0004

k-NN · [0.0004, 0.0004]

Model interpretation

Feature importance and SHAP values

94%

of Gradient Boosting's impurity importance is **Workload** (96% in Random Forest).

- **Age** and **Heart_Rate** are far behind; Duration and Body_Temp are close to zero.
- **SHAP agrees**: rankings shift by at most one place, except Height in GB (two).



Final model

Tuned Gradient Boosting (400 trees, depth 5, learning rate 0.1) performs best on every metric.

0.9995

R^2 on the test set

1.39

RMSE, kcal

0.96

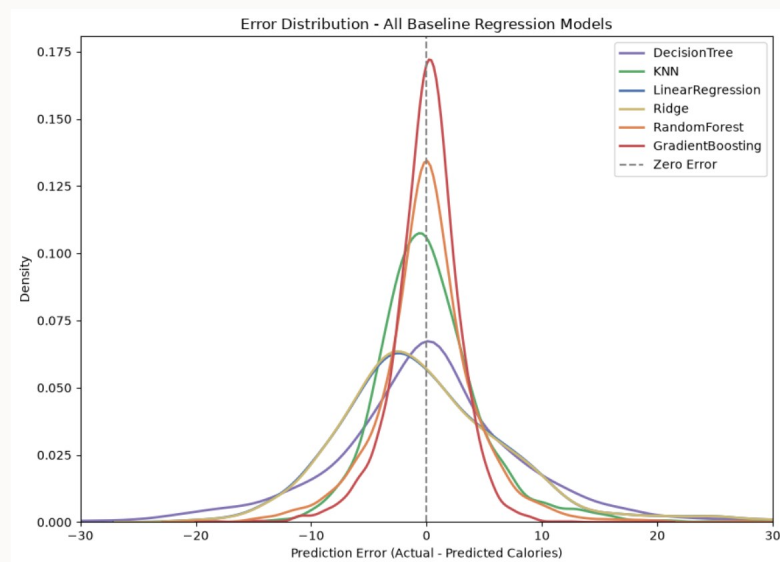
MAE, kcal (range 1–314)

Why not the ensemble?

Voting ensemble (RF + GB + k-NN): R^2 0.9988, MAE 1.51. Equal-weight averaging with weaker models dilutes the best one.

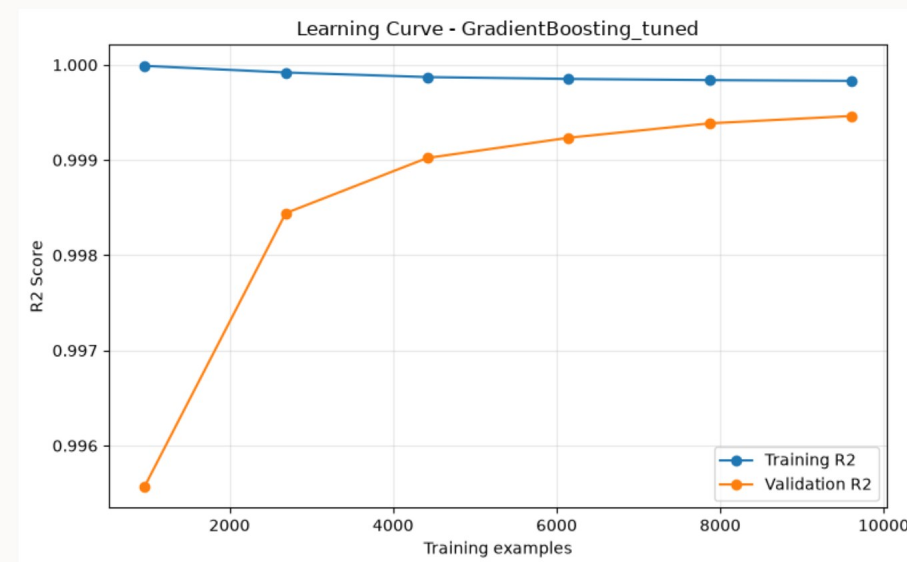
Diagnostics

Prediction errors



GB has the narrowest peak at zero; linear models are wide and shifted, a sign of missed curvature.

Learning curve



Validation R^2 rises from about 0.9956 to 0.9995 and the gap narrows: no sign of overfitting.

Results and conclusion

Baselines

GB is best (R^2 0.9979), ahead of RF and k-NN; linear models are clearly behind. 5-fold CV gives the same ranking.

Tuning

Improved every tuned model; the gains are consistent across 10 random splits.

No overfitting

Learning curve: validation R^2 rises to 0.9995 and the gap to training closes.

Importance and SHAP

Workload drives the predictions; Age and Heart_Rate are far behind. Both methods agree.

Voting ensemble

R^2 0.9988, MAE 1.51: worse than tuned GB alone.

Recommendation

Tuned Gradient Boosting: R^2 0.9995, RMSE 1.39, MAE 0.96 kcal.

Thank you
